

TECHNICAL DESIGN DOCUMENT

# InkWell Blogging Platform

Full-Stack System Architecture and Implementation Specifications

|             |                                     |
|-------------|-------------------------------------|
| Author:     | Saurabh Nagaich                     |
| Frameworks: | Angular 17, ASP.NET Core 8 (.NET 8) |
| Database:   | SQL Server (Database-per-Service)   |

## 1. System Architecture Overview

I structured the InkWell blogging platform using a decoupled **Microservices Architecture**. The core idea is to separate business logics into independent service modules. Rather than letting the frontend talk directly to multiple internal services, I placed a centralized gateway in the middle to handle routing, security, and traffic dispatching.

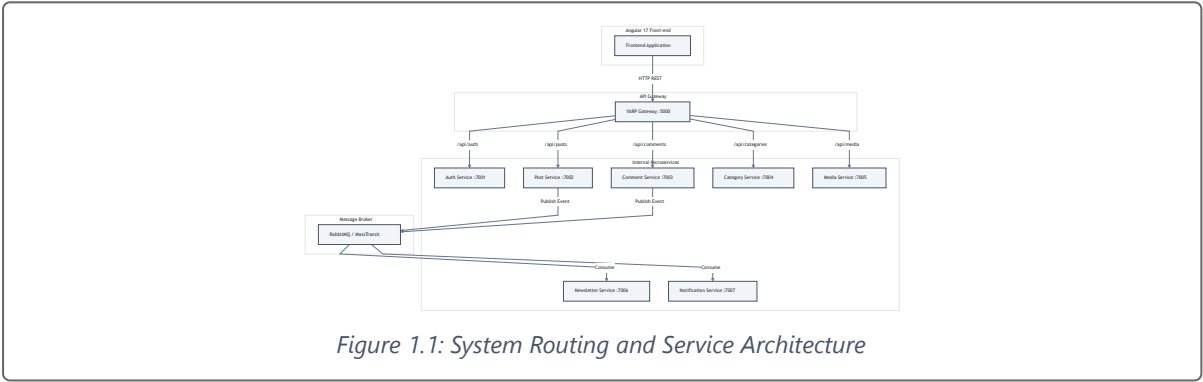


Figure 1.1: System Routing and Service Architecture

### MY DESIGN DECISION: USING YARP API GATEWAY

I integrated **YARP (YET ANOTHER REVERSE PROXY)** as the central gateway of the system. I chose it because it is built natively on .NET 8, making it extremely fast. YARP listens on port 5000 and parses the URL request prefixes. If it sees `/api/auth`, it redirects the call to the Auth Service on port 7001. This approach hides all microservice ports from the outside world and completely avoids CORS errors.

### 1.1. Why I chose a Microservice Approach

A monolith codebase gets bloated and hard to maintain over time. For instance, if the comments section experiences a heavy traffic spike, it shouldn't slow down the login service. By breaking the system into microservices, each service works inside its own scope. If the comments service goes down, readers can still log in and write posts without any issues.

### 1.2. How the Services Communicate

For standard frontend actions, like fetching post feeds or checking user credentials, the frontend uses direct synchronous HTTP calls. But for background tasks (like updating analytics or notifying active subscribers), synchronous REST calls would make the UI laggy. To solve this, I set up **RabbitMQ with MassTransit**. The posting service publishes a simple message to the broker and immediately responds to the user, while downstream services process the background tasks asynchronously.

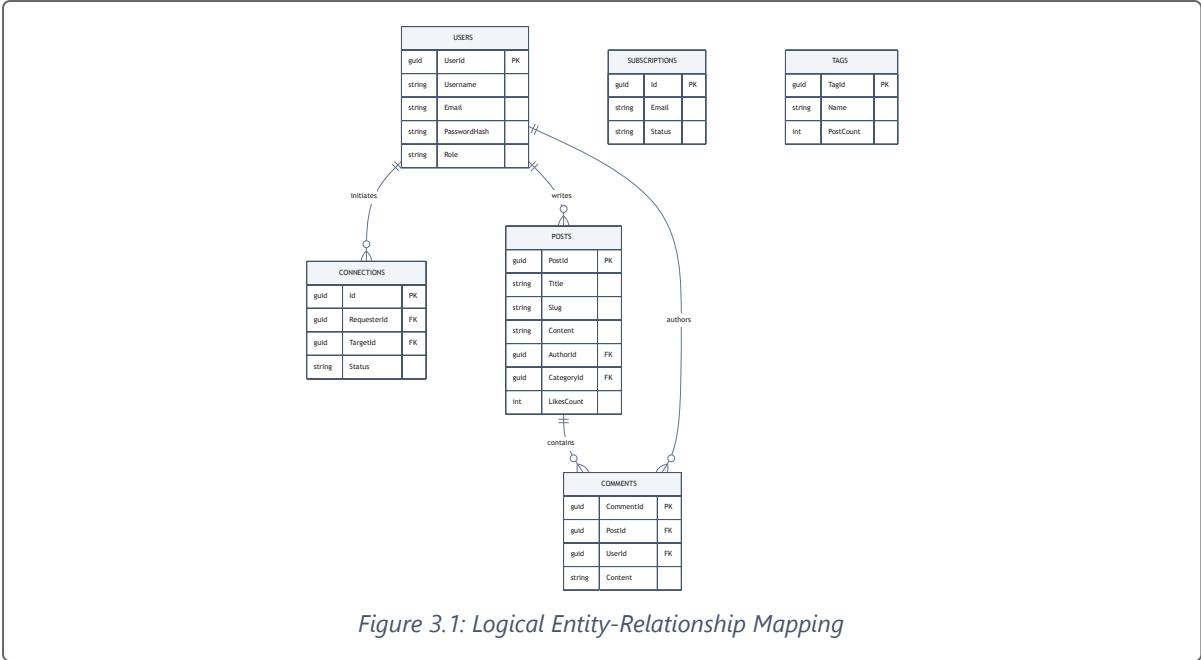
## 2. Service Port Allocations

To avoid local address conflicts and organize testing registries, I mapped each microservice to a dedicated runtime port. The gateway routes requests based on this layout:

| Service Layer        | Runtime Port | Tech & Tools Used     | Primary Responsibility                                     |
|----------------------|--------------|-----------------------|------------------------------------------------------------|
| Frontend SPA         | Port :4200   | Angular 17, RxJS, CSS | Serves the user views and processes state.                 |
| API Gateway          | Port :5000   | YARP, ASP.NET Core 8  | Acts as the single entrance, proxies routes internally.    |
| Auth Service         | Port :7001   | C#, .NET 8, EF Core 8 | Handles signup, logins, BCrypt hashing, and JWT tokens.    |
| Post Service         | Port :7002   | C#, .NET 8, EF Core 8 | Manages article composing, editing, slug logic, and likes. |
| Comment Service      | Port :7003   | C#, .NET 8, EF Core 8 | Manages commenting threads and dynamic replies.            |
| Category Service     | Port :7004   | C#, .NET 8, EF Core 8 | Manages dynamic hashtags, tags cataloging, and slugs.      |
| Media Service        | Port :7005   | C#, .NET 8, EF Core 8 | Manages file uploads, post banners, and author avatars.    |
| Newsletter Service   | Port :7006   | C#, .NET 8, EF Core 8 | Tracks subscriptions and triggers newsletters.             |
| Notification Service | Port :7007   | C#, .NET 8, EF Core 8 | Tracks reader activity logs and system notifications.      |

## 3. Database Design & Logical Relations

I followed a strict **Database-per-Service** pattern. Each service has its own dedicated MS SQL Server database. This keeps the data fully isolated and ensures that no service can execute direct queries on another service's tables.



MY DESIGN DECISION: RESOLVING CROSS-DATABASE RELATIONS

Since physical database-level foreign key joins are not possible across separate databases, I used **\*\*Logical Relations\*\***. For example, the Posts table inside PostDb holds an AuthorId column which is a simple GUID. This GUID references the UserId in the isolated AuthDb. When displaying a post with its author's profile, the frontend handles data aggregation by sending fast, parallel HTTP calls to both the Post and Auth services, combining the data on the client side.

3.1. Detailed Database Table Layout

3.1.1. Auth Service Database (`InkWell\_AuthDb`)

- **Users Table:** Stores user profiles and roles. Contains columns: UserId (GUID), Username, Email, PasswordHash (BCrypt hashed string), and Role (Reader/Author/Admin).
- **Connections Table:** Manages follow relationships. Contains: RequesterId (GUID), TargetId (GUID), and Status (Pending/Accepted).

3.1.2. Post Service Database (`InkWell\_PostDb`)

- **Posts Table:** Holds main story details. Contains: PostId (GUID), Title, Slug, Content, LikesCount (Int), and logical foreign key AuthorId.

3.1.3. Comment Service Database (`InkWell\_CommentDb`)

- **Comments Table:** Discussion data. Holds CommentId, PostId (Logical ref), UserId (Logical ref), and the comment text Content.

3.1.4. Category Service Database (`InkWell\_CategoryDb`)

- **Categories & Tags:** Manages dynamic cataloging. Tags table tracks TagId, Name, and PostCount.

4. Frontend Angular Module Routing Design

I designed the Angular 17 frontend utilizing highly organized modules. To keep the initial page load time as fast as possible, I structured the primary feature routes to load dynamically.

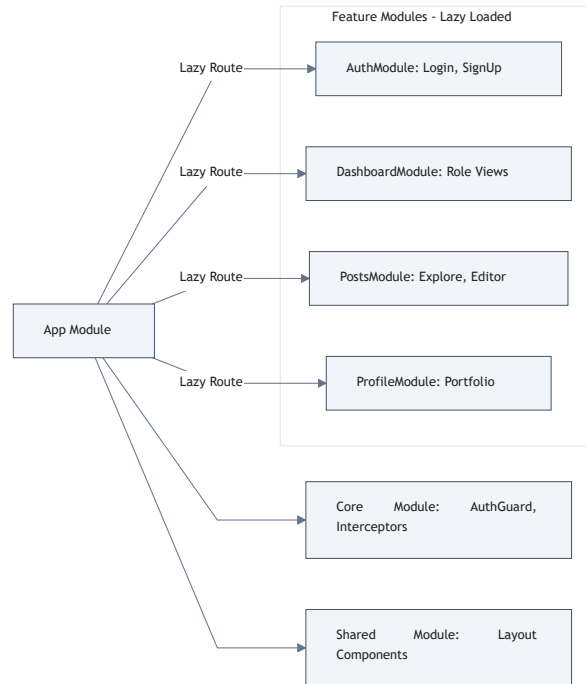


Figure 4.1: Frontend Angular Architecture Map

### MY DESIGN DECISION: OPTIMIZING WITH LAZY LOADING & INTERCEPTORS

I used Angular's `loadChildren` routing rules to split the app bundle. Instead of downloading all scripts at once, the browser only downloads the specific code (like the post editor) when the user navigates there. To secure API requests, I wrote a global `JwtInterceptor` inside the Core module. It automatically intercepts all HTTP calls and attaches the user's JWT token (retrieved from `LocalStorage`) to the header as a Bearer authorization token.

#### 4.1. Access Guard Routing

To lock down dashboard pages and the compose editor from guests, I engineered a global `AuthGuard`. This guard checks the decrypted token claims in the browser. If a non-authorized reader attempts to navigate to the `/admin` dashboard, the guard intercepts the routing event and redirects them to a secure Access Denied warning view.

## 5. Event-Driven Workflow Example

To showcase how the front-end, backend services, and message broker execute complex workflows, here is the detailed sequence flow that triggers when a reader likes an article:

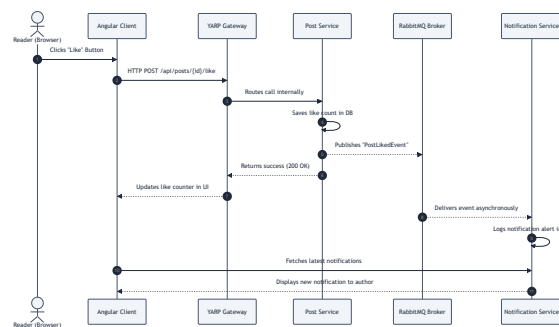


Figure 5.1: Real-Time Event-Driven Likes Pipeline

WHY I DESIGNED THIS FLOW

This event-driven flow ensures that the user's browser doesn't hang. When the user likes a post, the Post Service performs the quick database update and immediately fires a `PostLikedEvent` to RabbitMQ. The server returns a `200 OK` response to the frontend in milliseconds, updating the UI counter instantly. The task of logging the alert is handled completely offline in the background by the Notification Service, keeping the client thread light and fast.

6. Quality Gates & Continuous Testing

To ensure that the application is production-ready and free of security vulnerabilities, I integrated static analysis tools and set up automated test gates throughout the codebase development.

6.1. Quality Control Matrices

| Quality Gate Category         | Testing Tools     | Target Goal         | Project Purpose                                                    |
|-------------------------------|-------------------|---------------------|--------------------------------------------------------------------|
| Static Code Scans             | SonarQube Scanner | Quality Gate: PASS  | Scans for security vulnerabilities, bugs, and complex code smells. |
| Backend Logic Validation      | MSTest Suite      | > 80% Code Coverage | Tests business logic, entity mappings, and controllers.            |
| Frontend Component Validation | Jasmine & Karma   | > 75% Code Coverage | Mocks API calls, tests component bindings and guards.              |
| Browser End-to-End Testing    | Playwright E2E    | 100% Pass Rate      | Tests dynamic logins, editing, and searches across browsers.       |

6.2. Static Code Auditing via SonarQube

I set up **SonarQube Scanner** in the project to maintain code quality. The scanner performs a semantic pass of both the C# backend and TypeScript frontend codebases, identifying potential bugs (like unhandled null pointers) and security issues (such as insecure cryptographic keys or unvalidated inputs) early in the development cycle, ensuring that the final build is highly clean and production-ready.